TASK 6:

Form Code:

app > main.py > ...

You, 36 minutes ago | 1 author (You)

```python
import sqlite3
from tkinter import *
from datetime import date
from app.ticket import Ticket, generate_ticket_id

def save_ticket(student_number, student_name, category, priority, description, status):
    # Validation rules
    if not student_number.startswith("ST"):
        error_label.config(text="Error: Student number must start with 'ST'")
        return
    if len(description) < 10 or len(description) > 200:
        error_label.config(text="Error: Description must be between 10 and 200 characters")
        return
    if not student_name.strip():
        error_label.config(text="Error: Student name cannot be empty")
        return

    # Generate ticket ID and date
    ticket_id = generate_ticket_id()
    new_ticket = Ticket(ticket_id, student_number, student_name, category,
                        description, priority, status, str(date.today()), "")

    # Save to SQLite
    conn = sqlite3.connect("data/tickets.db")
    cursor = conn.cursor()
    cursor.execute("""
        INSERT INTO tickets (ticket_id, student_number, student_name, category, description, priority, status, date_logged, closing_comm
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
    """, (new_ticket.ticket_id, new_ticket.student_number, new_ticket.student_name,
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

```python
    # Save to SQLite
    conn = sqlite3.connect("data/tickets.db")
    cursor = conn.cursor()
    cursor.execute("""
        INSERT INTO tickets (ticket_id, student_number, student_name, category, description, priority, status, date_logged,
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
    """, (new_ticket.ticket_id, new_ticket.student_number, new_ticket.student_name,
          new_ticket.category, new_ticket.description, new_ticket.priority,
          new_ticket.status, new_ticket.date_logged, new_ticket.closing_comment))
    conn.commit()
    conn.close()

    success_label.config(text=f"Ticket {ticket_id} saved successfully!")
    error_label.config(text="")

# Tkinter form
root = Tk()
root.title("Add Ticket")

Label(root, text="Student Number").grid(row=0, column=0)
student_number = Entry(root)
student_number.grid(row=0, column=1)

Label(root, text="Student Name").grid(row=1, column=0)
student_name = Entry(root)
student_name.grid(row=1, column=1)

Label(root, text="Category").grid(row=2, column=0)
category = Entry(root)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS app > main.py > ...

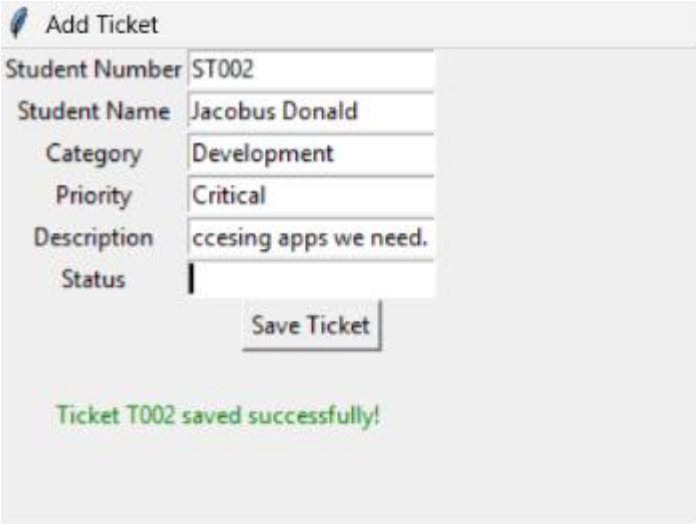
```python
Label(root, text="Student Name").grid(row=1, column=0)
student_name = Entry(root)
student_name.grid(row=1, column=1)

Label(root, text="Category").grid(row=2, column=0)
category = Entry(root)
category.grid(row=2, column=1)

Label(root, text="Priority").grid(row=3, column=0)
priority = Entry(root)
priority.grid(row=3, column=1)

Label(root, text="Description").grid(row=4, column=0)
description = Entry(root)
description.grid(row=4, column=1)
        You, 36 minutes ago • Uncommitted changes
Label(root, text="Status").grid(row=5, column=0)
status = Entry(root)
status.grid(row=5, column=1)

Button(root, text="Save Ticket", command=lambda: save_ticket(
    student_number.get(), student_name.get(), category.get(),
    priority.get(), description.get(), status.get()
)).grid(row=6, column=1)

# Error and success labels must be defined before use
error_label = Label(root, text="", fg="red")
error_label.grid(row=7, column=0, columnspan=2)

success_label = Label(root, text="", fg="green")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

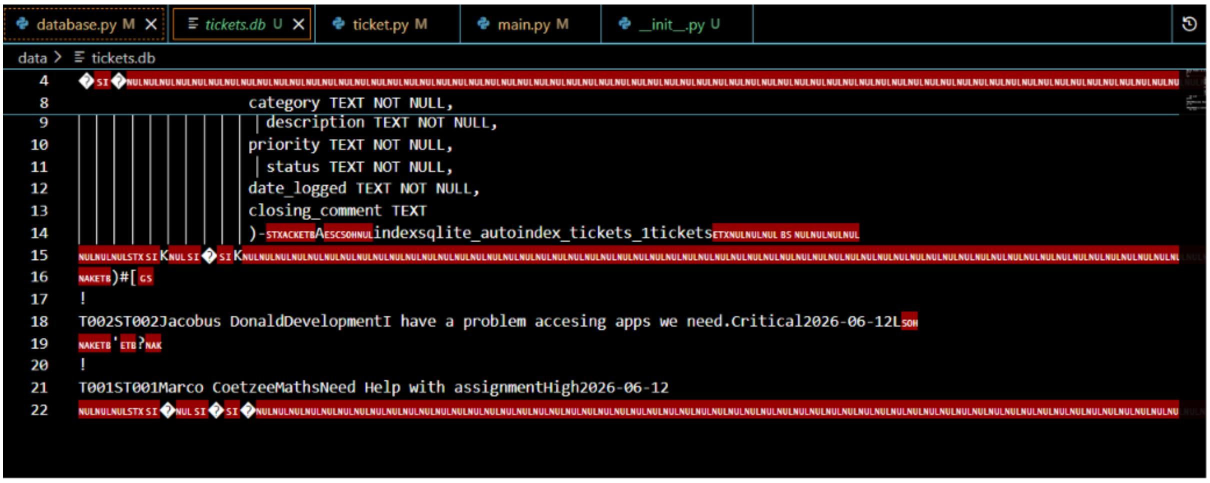
app > main.py > ...

```python
Button(root, text="Save Ticket", command=lambda: save_ticket(
    student_number.get(), student_name.get(), category.get(),
    priority.get(), description.get(), status.get()
)).grid(row=6, column=1)

# Error and success labels must be defined before use
error_label = Label(root, text="", fg="red")
error_label.grid(row=7, column=0, columnspan=2)

success_label = Label(root, text="", fg="green")
success_label.grid(row=8, column=0, columnspan=2)

root.mainloop()

conn = sqlite3.connect("data/tickets.db")
cursor = conn.cursor()

cursor.execute("SELECT * FROM tickets")
rows = cursor.fetchall()

for row in rows:
    print(row)

conn.close()
```

Form:



Results:



Results strore in data/tickets